COSC3106: Theory of Computing

Lec#5: Equivalence of DFAs and NFAs

School of Computer Science and Technology

This lecture

- By the end of this lecture, you will learn …
  - How to convert NFA to DFA

- **Equivalence of DFAs and NFAs**
  - **NFA:** $N = \{Q, \Sigma, \delta, q, F\}$

$$\delta : Q \times \Sigma_\varepsilon \to P(Q) \; ; \; \Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$$

  - **DFA:** $M = \{Q', \Sigma, \delta', q', F'\}$

$$\delta' : Q' \times \Sigma \to Q'$$

- **Converting NFA to Equivalent DFA**
  - **General Approach:**
    - **When an NFA reads part of an input string and has not yet processed the rest, we ask the following question:**
      - **What are the possible states that the NFA could be in at this point?**
      - **The key idea in converting an NFA to a DFA is that the DFA explicitly keeps track of all possible states the NFA could be in simultaneously.**

- Easy Case: No ε-transitions in NFA "N"
  - DFA construction: $M = \{Q', \Sigma, \delta', q', F'\}$
  - NFA Construction: $N = \{Q, \Sigma, \delta, q, F\}$
  - States: The set of states of DFA will be the power set of states of NFA

$$Q' = P(Q) = \{R : R \subseteq Q\}$$

    - For every subset R of the set of states Q, there is one state in DFA, this means if NFA has 5 states the power set has size of $2^5 = 32$ states

  - Start State: $q' = q$; (Same as the NFA start state.)
  - Accept States: $F' = \{R : R \subseteq Q, R \cap F \neq \phi\}$
    - Any DFA state that contains at least one NFA accept state is an accept state.
  - Transition Function:

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a)$$

Converting NFAs to Equivalent NFAs

- **Easy Case: No ε-transitions in NFA "N"**
  - **Ex:**

- General Case: ε-transitions in NFA "N"
  - NFA process a string:
    - Start in q, zero or more ε-transitions
    - Read one symbol and go to the next state
    - Zero or more ε-transitions
    - Read one symbol and go to the next state
    - Zero or more ε-transitions
  - ε-closure:
    - For $r \in Q$ of NFA, $C_\varepsilon(r)$ is a set of all states reachable from state "r" by making zero or more ε-transitions.

- **General Case:**
  - **Ex:**

- General Case: Now we can define the equivalent DFA "M"
  - DFA construction: $M = \{Q', \Sigma, \delta', q', F'\}$
  - States:

$$Q' = P(Q)$$

    - Each DFA state corresponds to a subset of NFA states.
    - The DFA keeps track of "all possible states" the NFA could be in after reading some input.
  - Start State:

$$q' = q$$

  - Accept States:

$$F' = \{R \subseteq Q, R \cap F \neq \phi\}$$

    - Any DFA state that contains at least one NFA accept state is an accept state.
  - Transition Function:

$$\delta' : Q' \times \Sigma \to Q'$$

$$\delta'(R, a) = \bigcup_{r \in R} \bigcup_{s \in \delta(r,a)} C_\varepsilon(r)$$

    - In words: from each NFA state in R, follow a-transitions; collect all reachable states including ε-transitions.

Closure under the regular operations

- **Theorem 2.1:**
  - **A language A is regular if and only if there exists a nondeterministic finite automaton NFA that accepts A.**

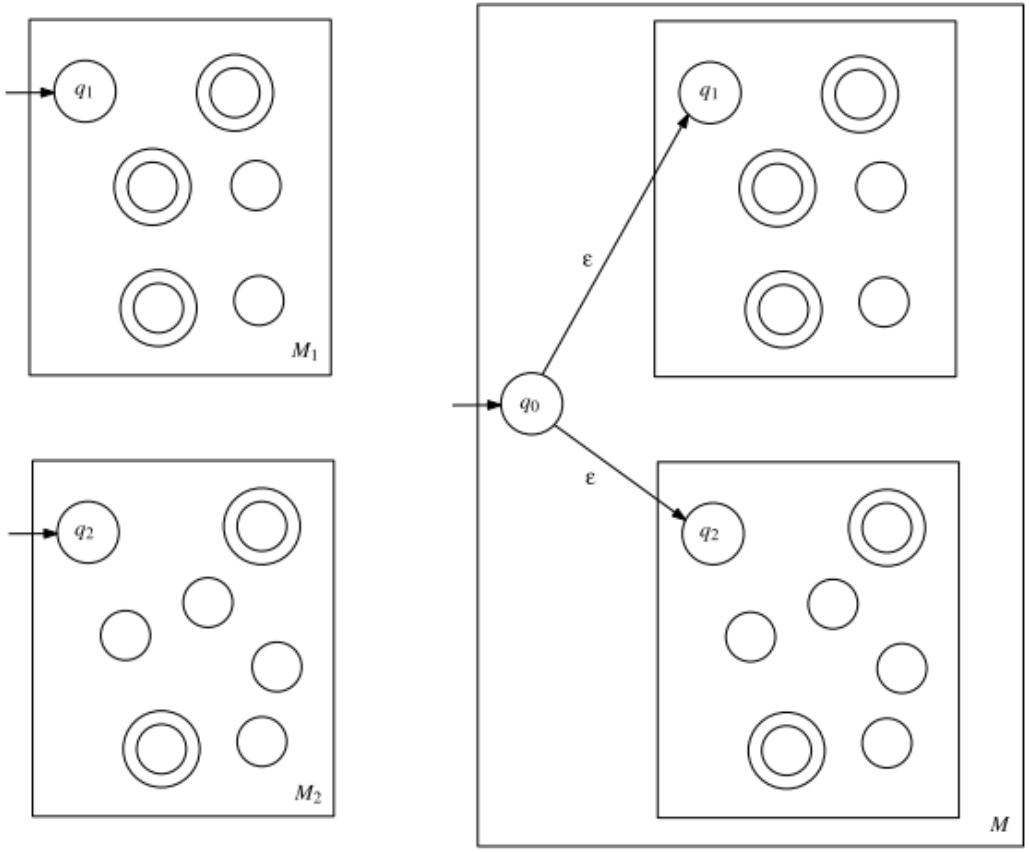
- **We have explained why DFA makes it not clear that the concatenation of two regular languages is regular, and that the star of a regular language is regular.**

Closure under the regular operations

- Now, we will see that the concept of NFA can be used to give a simple proof of the fact that the regular languages are indeed closed under the regular operations.

- Theorem 2.2:
  - The set of regular languages is closed under the union operation
  - i.e., if A_1 and A_2 are regular languages over the same alphabet Σ, then $A_1 \cup A_2$ is also a regular language.

- Proof:



Closure under the regular operations

- Theorem 2.1:
  - A language A is regular if and only if there exists a nondeterministic finite automaton NFA that accepts A.

- We have explained why DFA makes it not clear that the concatenation of two regular languages is regular, and that the star of a regular language is regular.

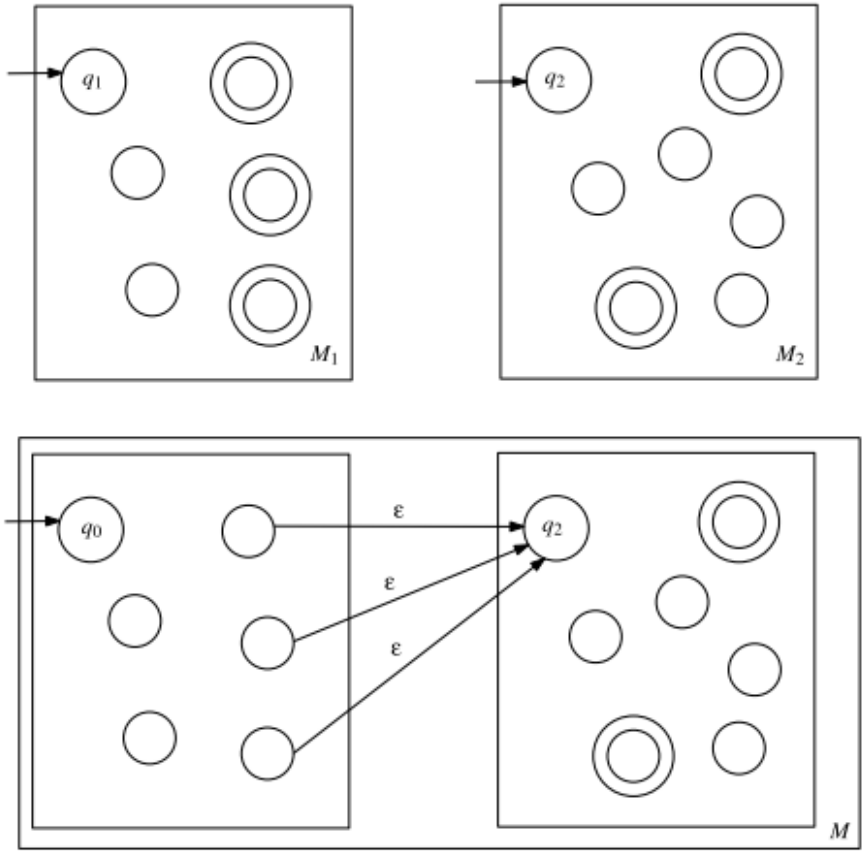
- Now, we will see that the concept of NFA can be used to give a simple proof of the fact that the regular languages are indeed closed under the regular operations.

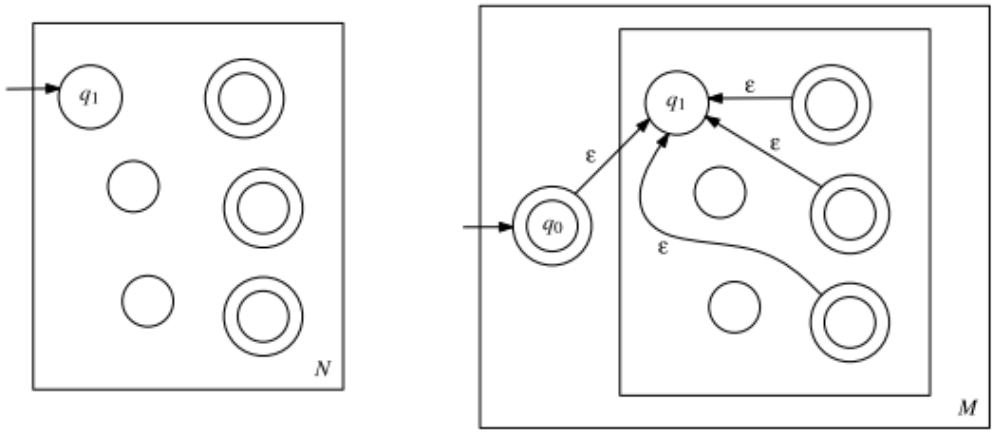
- Theorem 2.2:
  - The set of regular languages is closed under the union operation
  - i.e., if A_1 and A_2 are regular languages over the same alphabet Σ, then $A_1 \cup A_2$ is also a regular language.

- Proof:

- **Theorem 2.3:**
  - The set of regular languages is closed under the concatenation operation,
  - i.e., if A_1 and A_2 are regular languages over the same alphabet Σ, then A_1A_2 is also a regular language.

- **Proof:**



- **Theorem 2.4:**
  - **The set of regular languages is closed under the star operation,**
  - **i.e., if A is a regular language, then A^* is also a regular language.**

- **Proof:**